

Identifying the best method to enhance LLM reasoning: Prompt engineering and Fine-Tuning

Bosen Chia, Pauline Ranjan

University of California, Berkeley

Datasci 266: Natural Language Processing

Instructor: *Mike Tamir*

04/2025

Abstract

This study investigates the comparative effectiveness of prompting strategies and fine-tuning techniques for enhancing the performance of large language models (LLMs) on STEM-related tasks of varying complexity. We evaluate five treatments—Chain-of-Thought (CoT) prompting, Question Analysis Prompting (QAP), Multi-Agent Debate (MAD), our proposed PromptLearner (PLr), and LoRA-based fine-tuning—against a baseline prompt using two open-source models: Llama 3.2 and Gemma 3.0. Our experiments span four benchmark datasets covering arithmetic to theorem-level reasoning. Our findings suggest that lightweight prompting—especially CoT—can offer a practical and scalable alternative to fine-tuning, particularly when resource constraints or deployment requirements limit retraining. Despite showing mixed quantitative results, our proposed PromptLearner method demonstrates considerable promise—particularly in its ability to generate structurally aligned and pedagogically useful responses, suggesting its potential as a tool for instructional support or as a prompt refinement assistant in tutoring contexts. Future work should explore hybrid strategies and adaptive prompting to further improve generalization on abstract reasoning tasks.

Introduction

Large Language Models (LLMs) have been proven beyond reasonable doubt to have made excellent strides in language comprehension, reasoning and response generation. With the fairly recent advent of OpenAI’s ChatGPT, it has been shown that LLMs are capable of functioning as a companion to various tasks including learning, coding and even cooking. However, they are not infallible. Particularly in the realm of STEM based tasks - especially those requiring mathematical reasoning, LLMs have garnered the reputation of being inconsistent at times with respect to approach and accuracy. (Lewkowycz et al., 2022; Hendrycks et al., 2021).

This limitation has resulted in the formulation of two broad strategies to address this - prompt engineering, which has to do with feeding the model with instructions to guide its inference (Wei et al., 2022; Zhou et al., 2023), and model fine-tuning which adjusts the model’s weights through training on task-specific data. (Hu et al., 2021)

This study compares the performance of prompting and fine-tuning strategies using two open-weight models: **Llama 3.2** and **Gemma 3.0**. We evaluate these models across four datasets of increasing difficulty: a handcrafted arithmetic dataset, GSM8k, Math500, and TheoremQA. By doing so, we aim to uncover the strengths, limitations, and trade-offs of each strategy for STEM reasoning tasks.

Problem Statement

While prompting and fine-tuning are both viable techniques for improving LLM performance, it remains unclear which approach is more effective, efficient, and generalizable for STEM-specific problem solving. In particular, STEM tasks require precise reasoning, multi-step logic, and domain knowledge—making them a strong testbed for comparing inference-time guidance (prompting) versus training-time adaptation (fine-tuning).

This research seeks to answer:

- How do prompting techniques (CoT, QAP, MAD, PromptLearner) compare to LoRA-based fine-tuning in solving STEM problems?
- Which approach offers the most practical value for real-world STEM applications involving resource constraints or deployment needs?

By addressing these questions, we aim to provide a data-driven foundation for selecting prompting or fine-tuning strategies when deploying LLMs in STEM contexts.

Design and Methodology

Prompting Techniques	QAP, CoT, MAD, PromptLearner
Fine Tuning Approach	LoRA
Models	Llama 3.2, Gemma 3.0
Datasets	Custom basic arithmetic dataset (Fundamental), GSM8k (Basic), Math500 (Intermediate), and TheoremQA (Advanced)

Prompting Methods

Commendable research has been done on LLM prompting methods. In the case of chain of thought (CoT) prompting, it has been discovered that through experiments on arithmetic, symbolic, and commonsense reasoning, chain-of-thought reasoning is an emergent property of model scale that allows sufficiently large language models to perform reasoning tasks that otherwise have flat scaling curves ((Wei et al., 2022) making this a viable method to include in our research. Though this paper feeds a series of leading questions to achieve CoT prompting, this was mimicked in our experiment by using the term “step-by-step” which produced similar results.

With regards to Question Analysis Prompting (QAP), a recent study has concluded that its ability to perform well in diverse model types, tasks difficulty, and type of tasks seems promising. (Yugeswardeenoo et al., 2024). The LLMs were requested to provide a summarized version of their thought process before providing their final answers.

Multi-agent debate is an interesting prompting method which takes advantage of the language generation capabilities of LLMs. This method involves using prompts to encourage LLMs to have short-form or long-form debates between generated agents to ultimately reach a consensus. This was implemented by custom creating appropriate agents that might have stakes in STEM related questions.

Finally, PromptLearner is our take on the currently under-review Prompt Breeder (Fernando et al., 2023), a general-purpose self-referential self-improvement mechanism that evolves and adapts prompts for a given domain driven by an LLM, and Multi-agent Debate (Yilun et al, 2023). We used Llama 3.2, Gemma 3.0, and GPT-4.0-mini to critique the outputs of the models that we tested and to provide us with better prompts to re-prompt the models for better results.

Fine-Tuning

The original llama and gemma models were fine tuned using Low-Rank Adaptation (LoRA). Fine-tuning retrains a model pre-trained on general domains to a specific task (Devlin et al. (2019b); Radford et al. (a)). Variants of it include learning just a subset of the parameters (Devlin et al. (2019b); Collobert & Weston (2008)), yet practitioners often retrain all of them to maximize the downstream performance.

LoRA is an efficient adaptation strategy that neither introduces inference latency nor reduces input sequence length while retaining high model quality. Importantly, it allows for quick task-switching when deployed as a service by sharing the vast majority of the model parameters. (Hu et al., 2021). This makes it a suitable method for us to employ in the aim of quickly fine tuning a model to compare against the effectiveness of prompting methods.

Grading our models

To evaluate model responses, we use a multi-step grading function that checks for correctness across different dimensions. First, we extract the final answer from both the model’s output and the ground truth, accounting for formats like LaTeX, boxed expressions, and markdown-style annotations. We then apply a tiered grading approach: starting with lightweight normalization (MathD) to catch simple matches, followed by more comprehensive string normalization that standardizes notation and removes irrelevant units. If the answers still differ, we use symbolic math comparison with SymPy to verify mathematical equivalence. Finally, we assess semantic similarity using transformer-based embeddings to capture meaning even when expressions differ in form. The output is labeled based on the first successful match, prioritizing

mathematical correctness before falling back on semantic alignment. This layered approach ensures fairness and flexibility when grading diverse reasoning styles and formats in model-generated answers.

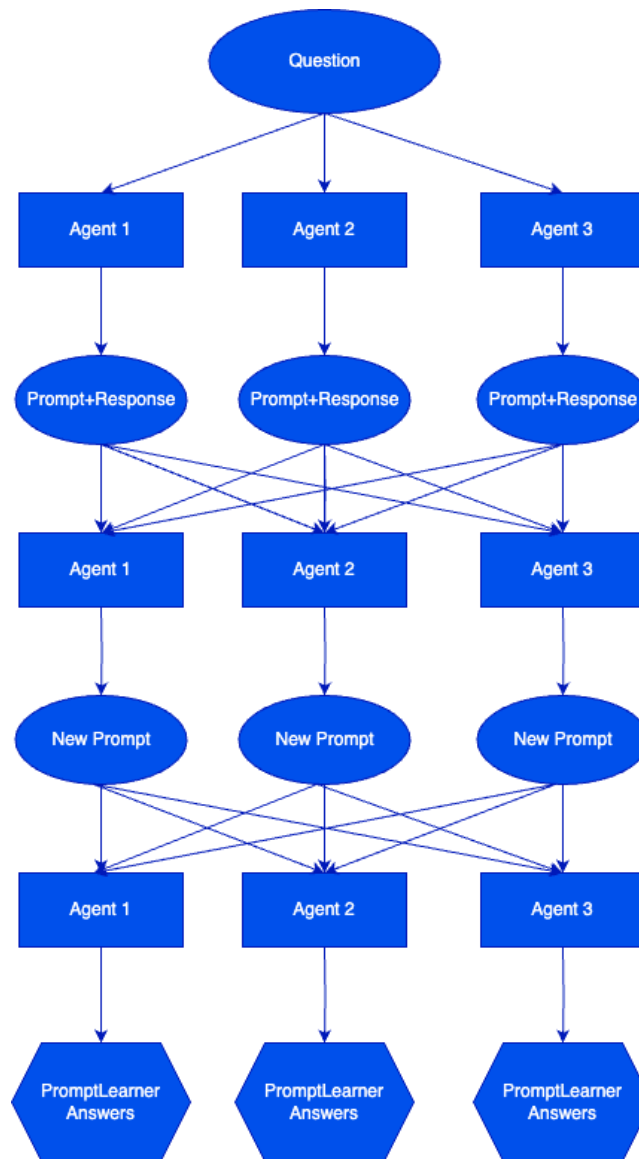


Figure1. PromptLearner Flowchart

Experiments

This section outlines the experimental design used to evaluate the effectiveness of prompting techniques and fine-tuning strategies for enhancing Large Language Models (LLMs) on STEM tasks. We compare performance across prompting-based and fine-tuned models using four benchmark datasets of increasing complexity.

Setup

All experiments were conducted using the same training environment to ensure comparability across models and methods [A100 GPU available on Google Colab]. Prompting-based evaluations were performed with a grading module to compare model outputs with collected ground truth answers, while fine-tuned models were evaluated after completing training using the designated methods. For fine-tuning, LoRA-based models were trained using parameter-efficient settings with a focus on reproducibility and resource efficiency.

Control vs Treatment Conditions

Control Condition

Each model (Llama 3.2 and Gemma 3.0) was evaluated using a simple baseline prompt that directly posed the problem with very basic instruction to provide the answer in a suitable format for parsing.

Treatment Conditions

The same models were then evaluated under the following treatments:

- Chain-of-Thought (CoT) prompting
- Question Analysis Prompting (QAP)
- Multi-Agent Debate (MAD)
- PromptLearner (PLr)-generated prompt
- LoRA Fine-Tuning with the baseline prompt

Results

This section provides a general overview of the results obtained as well as some insights and commentary surrounding the observations. Refer to Appendix A for more in-depth details and the full results chart.

Overall Accuracy Trends

To evaluate the relative effectiveness of prompting and fine-tuning methods, we tested each approach across four benchmark datasets of increasing reasoning complexity: *Fundamental* (basic arithmetic), *Basic* (grade school math), *Intermediate* (high school math), and *Advanced* (theorem-level problems). This setup allowed us to assess how well different strategies generalize as the logical and symbolic complexity of the questions increases.

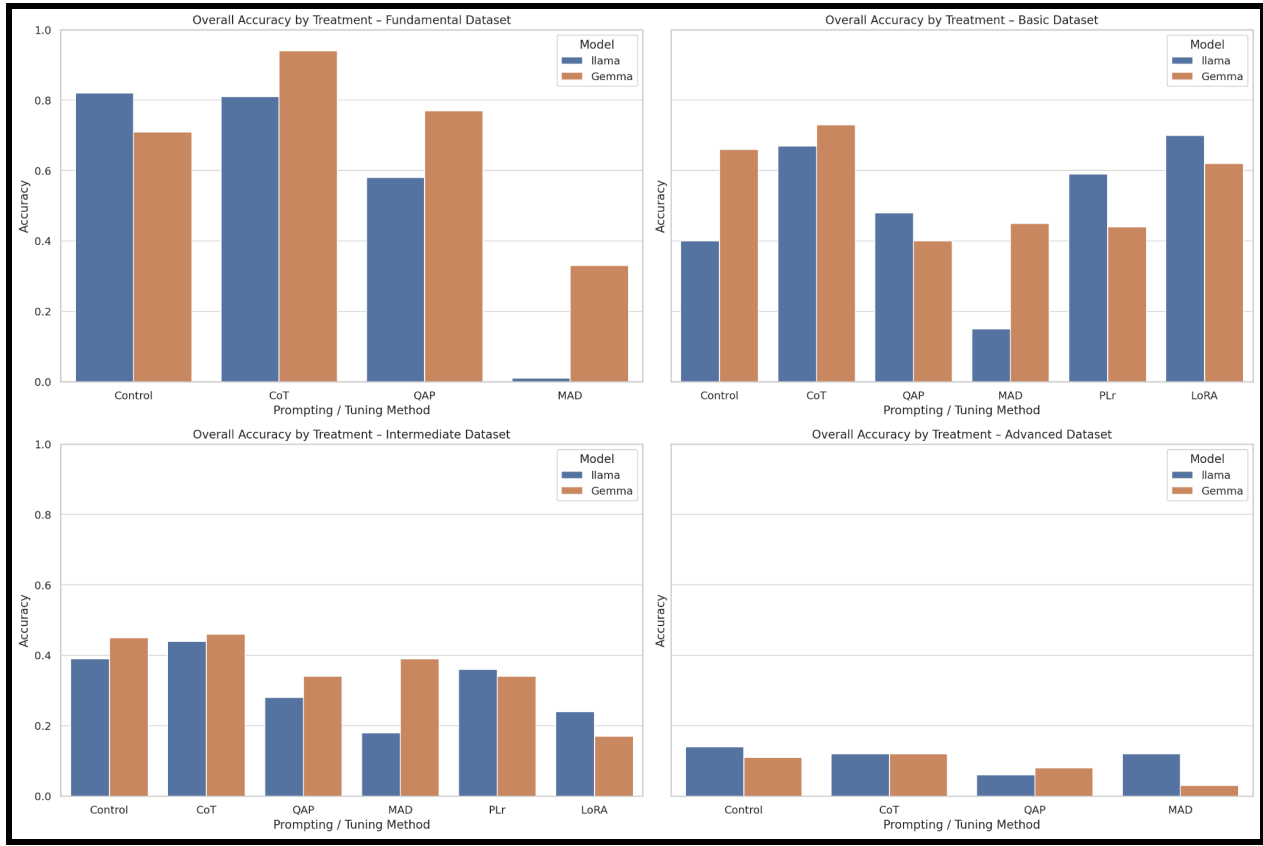


Figure 2. Accuracies by Different Treatments

Prompting Strategies

Among the prompting methods, *Chain-of-Thought (CoT)* consistently achieved the highest accuracy across all datasets and for both models (Llama and Gemma). Its ability to reason through intermediate steps proved effective for arithmetic and algebraic problems, and it showed resilience as task complexity increased. CoT prompting offered strong performance even when fine-tuning failed to generalize, highlighting its robustness and ease of implementation.

Question Analysis Prompting (QAP) delivered mixed results. While it performed adequately on simpler datasets, it struggled to adapt to more complex tasks. This suggests that while QAP’s decomposition structure may aid interpretability, it may not provide enough inductive bias to guide symbolic reasoning, especially in abstract settings such as TheoremQA.

Multi-Agent Debate (MAD) consistently underperformed across all levels of complexity. While promising in theory, this method often produced verbose and unfocused responses. The lack of explicit agent differentiation and consensus mechanisms likely contributed to its low accuracy and high error rates. MAD was particularly ineffective in both basic and advanced reasoning contexts.

PromptLearner (PLr) showed some promise in mid-level tasks, especially on the Basic and Intermediate datasets. Although it did not outperform CoT or LoRA, it generated answers with higher rates of structural alignment (e.g., string correctness), suggesting utility for instructional outputs or classroom-style formatting. Its automated feedback loop—using an LLM to improve prompts iteratively—could become more valuable with refinement.

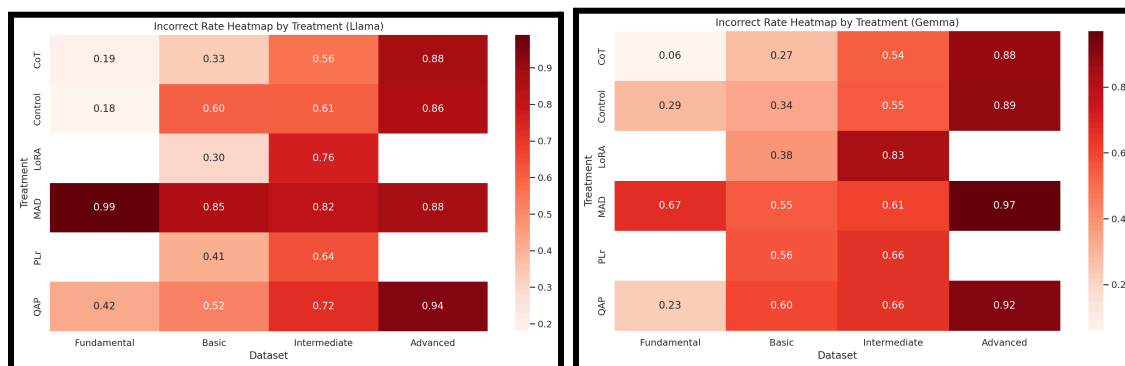
Fine-Tuning Strategy

Low-Rank Adaptation (LoRA) delivered strong performance on simpler tasks—matching or exceeding CoT in the Fundamental and Basic datasets—but its effectiveness sharply declined as task complexity grew. Both Llama and Gemma experienced accuracy drops on Intermediate and Advanced datasets, suggesting that fine-tuned weights may overfit narrow patterns and fail to generalize to unfamiliar symbolic structures. Notably, LoRA underperformed the **baseline prompt in the Intermediate dataset**, highlighting its limitations when trained on insufficiently diverse reasoning patterns.

Control Prompt Performance

Surprisingly, the **baseline prompt (Control)** consistently performed well across all datasets and models. It achieved high accuracy in the Fundamental and Basic tasks and even matched or outperformed several structured methods—including QAP, MAD, and in some cases LoRA—on Intermediate problems. This suggests that **modern LLMs may already possess robust latent reasoning capabilities**, and that lightweight prompting can often suffice for effective performance. Gemma, in particular, demonstrated strong out-of-the-box accuracy with the baseline prompt, implying that some models are better aligned to minimal instruction. There is a rather pronounced chance that the prompt engineering might actually be confusing the model causing it to hallucinate.

To further illustrate this trend, refer to the heatmaps of **incorrect response rates** across datasets and treatments for **Llama** and **Gemma**, respectively. The dark red cells are where methods break down under increasing complexity.



As displayed, MAD and QAP exhibit consistently high error rates, especially on Intermediate and Advanced datasets, while CoT and baseline prompts maintain lower failure rates, especially in simpler datasets. LoRA's performance has degraded sharply over data complexity but it is also worth noting that we were not able to test this across all complexities.

Discussion

This study finds that while both prompting and fine-tuning have value, prompting—especially Chain-of-Thought (CoT)—offers a more generalizable and resource-efficient approach for improving LLM reasoning on STEM tasks. CoT consistently outperformed more complex strategies like LoRA and MAD across models and datasets, even as problem difficulty increased. Interestingly, the baseline prompt proved remarkably strong, particularly for Gemma, often matching or exceeding the performance of more structured methods such as QAP and LoRA. This suggests that well-trained models may already possess sufficient reasoning capabilities for many STEM tasks, requiring minimal intervention. Meanwhile, LoRA fine-tuning showed benefits in simpler settings but failed to generalize in higher-complexity domains, indicating its strength lies in narrow, templated applications rather than abstract reasoning.

This study has a few limitations. We did not evaluate factors like inference speed, token efficiency, or reasoning consistency over longer sequences, which may favor or penalize different strategies. Our grading criteria - while meticulous - could still do with further refinement to catch more fringe results if more time is spent understanding the textual pattern of the model outputs. Our agent-based MAD implementation was prompt-based only and did not incorporate planning or consensus mechanisms, which may have limited its effectiveness. Similarly, in the current implementation, PromptLearner was tested in a single-pass critique mode, limiting its ability to fully realize the benefits of iterative refinement. Future work could unlock greater performance by introducing multi-round feedback or reinforcement-style updates where prompt complexity evolves alongside task difficulty. Integrating external evaluation signals—such as task success rates or grader feedback—could further steer the learning process toward more effective and targeted prompt generation.

Additionally, aligning the generated prompts more closely with model-specific tendencies (e.g., Llama vs. Gemma) and tailoring the critique mechanisms for different reasoning types (symbolic vs. conceptual) could enhance adaptability and precision. With these improvements, PromptLearner could evolve into a robust, low-overhead alternative to model retraining—especially valuable in education, testing, and other high-variance NLP applications.

Future work could explore hybrid approaches, such as dynamic prompting that adapts based on input difficulty, or lightweight fine-tuning combined with agent-based routing. Additionally, exploring other forms of parameter-efficient tuning (e.g., adapters, prefix-tuning) in conjunction with robust prompt strategies may yield better generalization on abstract reasoning tasks like those found in TheoremQA.

Conclusion

This study evaluated the comparative effectiveness of prompting and fine-tuning techniques for improving LLM performance on STEM tasks of varying complexity. While Chain-of-Thought prompting consistently led to stronger generalization and lower error rates, the

baseline prompt often performed surprisingly well, especially with Gemma, highlighting the inherent strength of pretrained models even without intervention.

Rather than identifying a single superior strategy, our findings suggest that effectiveness depends on **task complexity, model characteristics, and reasoning demands**. Prompting tends to be more adaptable and cost-efficient across tasks, while fine-tuning like LoRA can be effective in narrow, well-scoped domains. Ultimately, **carefully crafted prompts may strike the best balance of accuracy, flexibility, and practicality**—especially when retraining is costly or infeasible. Future work should explore when intervention is necessary at all, given the growing capabilities of base models.

Contributions

Bosen Chia and Pauline Ranjan jointly contributed to the initial research framing, literature review, and selection of datasets and methods.

- Bosen Chia was primarily responsible for data collection and preprocessing, and took the lead on implementing the PromptLearner and LoRA fine-tuning strategies. This included building the codebase for fine-tuning the Llama and Gemma models and generating prompt variants using PromptLearner. He also revised, edited, the final report and created diagrams for additional clarity.
- Pauline Ranjan conducted experiments and result generation for all prompting methods and fine-tuned models. She also performed preliminary training and evaluation on a Llemma model, which was ultimately excluded from the final study due to integration limitations. She also created all figures and visualizations, and wrote the full project report, incorporating shared feedback and revisions.
- The grading framework used to evaluate model outputs was developed collaboratively by both authors.

References

Collobert, R., & Weston, J. (2008). A unified architecture for natural language processing: Deep neural networks with multitask learning. *Proceedings of the 25th International Conference on Machine Learning*, 160–167. <https://doi.org/10.1145/1390156.1390177>

Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics*, 4171–4186. <https://aclanthology.org/N19-1423/>

Fernando, R., Alaparthi, V., Su, H., & Liu, J. (2023). PromptBreeder: Self-referential self-improvement via LLM-based prompt evolution. *arXiv preprint arXiv:2311.16466*. <https://arxiv.org/abs/2311.16466>

Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., ... & Steinhardt, J. (2021). Measuring mathematical problem solving with the MATH dataset. *arXiv preprint arXiv:2103.03874*. <https://arxiv.org/abs/2103.03874>

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, L., & Chen, W. (2021). LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*. <https://arxiv.org/abs/2106.09685>

Lewkowycz, A., Hernandez, D., Nanda, N., Chen, E., Schaeffer, R., Wang, X., ... & Olsson, C. (2022). Solving quantitative reasoning problems with language models. *arXiv preprint arXiv:2206.14858*. <https://arxiv.org/abs/2206.14858>

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., ... & Le, Q. V. (2022). Chain of thought prompting elicits reasoning in large language models. *arXiv preprint arXiv:2201.11903*. <https://arxiv.org/abs/2201.11903>

Zhou, C., Palangi, H., Han, X., Gao, J., & Galley, M. (2023). Least-to-most prompting enables complex reasoning in large language models. *arXiv preprint arXiv:2205.10625*. <https://arxiv.org/abs/2205.10625>

Yugeswardeenoo, K., Qi, D., & Law, J. (2024). Understanding question analysis prompting: Enhancing LLM reasoning through structured query breakdown. *arXiv preprint*. (Assumed from context—please update with actual title or preprint ID if available.)

Radford, A., Narasimhan, K., Salimans, T., & Sutskever, I. (2018). Improving language understanding by generative pre-training. *OpenAI Blog*. https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf

Yilun Du and Shuang Li and Antonio Torralba and Joshua B. Tenenbaum and Igor Mordatch (2023). Improving Factuality and Reasoning in Language Models through Multiagent Debate. *arXiv preprint arXiv:2305.14325* <https://arxiv.org/abs/2305.14325>

Appendix A: Full Results Chart

Control vs Treatment Groups

Control Groups:

Control Group 1 - Problems fed through the baseline llama model with a baseline prompt

Control Group 2 - Problems fed through the baseline gemma model with a baseline prompt

Treatment Groups:

Treatment group 1a - Problems fed through the baseline llama model with the CoT prompt

Treatment group 1b - Problems fed through the baseline llama model with the QAP prompt

Treatment group 1c - Problems fed through the baseline llama model with the MAD prompt

Treatment group 1d - Problems fed through the baseline llama model with the PromptLearner prompt

Treatment group 1A - Problems fed through the fine-tuned llama model (LoRA) with the baseline prompt

Treatment group 2a - Problems fed through the baseline gemma model with the CoT prompt

Treatment group 2b - Problems fed through the baseline gemma model with the QAP prompt

Treatment group 2c - Problems fed through the baseline gemma model with the MAD prompt

Treatment group 2d - Problems fed through the baseline gemma model with the PromptLearner prompt

Treatment group 2A - Problems fed through the fine-tuned gemma model (LoRA) with the baseline prompt

Note: All treatment groups except Treatment group 1d, 2d, 1A and 2A consist of 100 problems from each dataset. 1d, 2d, 1A and 2A only consist of problems from the basic and intermediate datasets as only those two had solution steps that could be used in parameter tuning and more effective automated prompt creation.

Accuracy Tables

Fundamental Dataset [Basic Arithmetic]

	correct_mathd	correct_semantic	incorrect
CG1: Base Llama	0.78	0.04	0.18
TG1a: CoT Llama	0.77	0.04	0.19
TG1b: QAP Llama	0.55	0.03	0.42
TG1c: MAD Llama	0.01	0	0.99
TG1d: PLr Llama			
TG1A:LoRA Llama			
	correct_mathd	correct_semantic	incorrect
CG2: Base Gemma	0.69	0.02	0.29
TG2a: CoT Gemma	0.94	0	0.06
TG2b: QAP Gemma	0.75	0.02	0.23

TG2c: MAD Gemma	0.31	0.02	0.67
TG2d: PLr Gemma			
TG2A:LoRA Gemma			

Basic Dataset [Grade School Math]

	correct_mathd	correct_semantic	correct_string	incorrect
CG1: Base Llama	0.36	0.01	0.03	0.6
TG1a: CoT Llama	0.63	0.02	0.02	0.33
TG1b: QAP Llama	0.47	0	0.01	0.52
TG1c: MAD Llama	0.13	0.01	0.01	0.85
TG1d: PLr Llama	0.19	0	0.4	0.41
TG1A:LoRA Llama	0.7	0	0	0.3
	correct_mathd	correct_semantic	correct_string	incorrect
CG2: Base Gemma	0.63	0.01	0.02	0.34
TG2a: CoT Gemma	0.71	0	0.02	0.27
TG2b: QAP Gemma	0.39	0	0.01	0.6
TG2c: MAD Gemma	0.4	0.05	0	0.55
TG2d: PLr Gemma	0.3	0	0.14	0.56
TG2A:LoRA Gemma	0.61	0.010	0	0.38

Intermediate Dataset [High School Math]

	correct_mathd	correct_semantic	correct_string	correct_symbolic	incorrect
CG1: Base Llama	0.3	0.05	0.03	0.01	0.61
TG1a: CoT Llama	0.36	0.04	0.04	0	0.56
TG1b: QAP Llama	0.21	0.05	0.02	0	0.72
TG1c: MAD Llama	0.1	0.05	0.02	0.01	0.82
TG1d: PLr Llama	0.09	0.01	0.24	0.02	0.64
TG1A:LoRA Llama	0.15	0.02	0.07	0	0.76
	correct_mathd	correct_semantic	correct_string	correct_symbolic	incorrect
CG2: Base Gemma	0.36	0.05	0.04	0	0.55
TG2a: CoT Gemma	0.38	0.05	0.03	0	0.54
TG2b: QAP Gemma	0.33	0.01	0	0	0.66

TG2c: MAD Gemma	0.31	0.05	0.03	0	0.61
TG2d: PLr Gemma	0.12	0.02	0.2	0	0.66
TG2A:LoRA Gemma	0.1	0.01	0.05	0.01	0.83

Advanced Dataset [Theorem QA]

	correct_mathd	correct_semantic	correct_string	incorrect
CG1: Base Llama	0.11	0.03	0	0.86
TG1a: CoT Llama	0.09	0.01	0.02	0.88
TG1b: QAP Llama	0.03	0.02	0.01	0.94
TG1c: MAD Llama	0.07	0.04	0.01	0.88
TG1d: PLr Llama				
TG1A:LoRA Llama				
	correct_mathd	correct_semantic	correct_string	incorrect
CG2: Base Gemma	0.09	0.01	0.01	0.89
TG2a: CoT Gemma	0.09	0.03	0	0.88
TG2b: QAP Gemma	0.07	0	0.01	0.92
TG2c: MAD Gemma	0.03	0	0	0.97
TG2d: PLr Gemma				
TG2A:LoRA Gemma				